

HW4 Reflection: Structured Workflow and TDD with Claude Code

Dhruv Gorasiya, Xuan Bai | CS7180: Vibe Coding | Northeastern University | March 2026

1. Workflow Comparison: Explore-Plan-Implement-Commit vs. Prior Practice

Prior to this assignment, our AI-assisted development workflow was largely reactive and unstructured. The OfficeHoursQ project, built using Cursor, serves as a direct point of comparison. That session had no exploration phase, no written plan, and no test coverage of any kind. The implementation log records seventeen steps, several dedicated entirely to recovery: the Cursor Write tool silently failed to persist files due to a workspace path containing spaces, the client directory was corrupted mid-session and had to be deleted and re-scaffolded from scratch, and the root git repository was lost and had to be reinitialized. The session concluded with an explicit note in the log: "no tests added." Every failure in that session was discovered reactively, after it had already propagated.

The Explore-Plan-Implement-Commit workflow applied in HW4 Part 2 addressed each of these failure modes directly. The Explore phase established verified ground truth before any code was written: when Claude Code confirmed all three backend directories contained only .gitkeep files, that finding was committed as a documented artifact. This single step prevented the most common failure mode from OfficeHoursQ, which was writing code against false assumptions about the existing codebase structure. The Plan phase then produced an 8-phase implementation contract written to tasks/todo.md before any implementation began, giving every subsequent step a clear definition of done that did not depend on the model's judgment in the moment.

The outcomes reflect the structural difference directly. The OfficeHoursQ session produced a working monorepo scaffold but left the codebase in a state with no automated verification. A single configuration change could silently break the application with no mechanism to detect it. The MockReady session ended Part 2 with 22 passing tests across the service and router layers, each encoding a specific behavioral assertion. The Commit discipline reinforced both: every commit in HW4 was scoped to a single decision, making the git log a traceable audit trail. In OfficeHoursQ, commits were coarse and infrequent, providing no visibility into how the codebase reached its final state.

2. Context Management Strategies

Managing context effectively across a multi-phase Claude Code session required deliberate strategies at each phase boundary. Three approaches made a measurable difference in session quality and output consistency.

- **Explicit stop gates between phases.** Rather than allowing Claude Code to chain operations autonomously, each phase ended with a manual verification step before the next prompt was issued. This kept the context window focused on one concern at a time and made failures immediately attributable to the most recent change. In the OfficeHoursQ session, chained operations meant that by the time a failure was detected, its origin was ambiguous. In HW4, every failure had a clear commit boundary.
- **TDD as a context anchor.** In Part 3, the red-green-refactor loop structured the context window as much as it enforced test discipline. By committing a failing test before requesting implementation, each implementation prompt arrived with a concrete, unambiguous success criterion already defined in the context. Claude Code did not need to infer what "done" meant. This eliminated the most common form of drift in AI-assisted sessions, where the model optimizes toward a plausible-looking output rather than a verifiable one. The explicit constraint "do not write any implementation yet" in every red-phase prompt was load-bearing: without it, Claude Code would co-generate tests and implementation together, producing tests that pass by construction rather than by correctness.
- **CLAUDE.md as a persistent context document.** The project's CLAUDE.md file encoded naming conventions, service layer patterns, and auth dependency injection details that would otherwise need to be restated in every prompt. This reduced prompt length, improved implementation consistency across all seven phases, and meant that code written in Phase 1 remained stylistically coherent with code written in Phase 7 without any additional coordination effort. In the OfficeHoursQ session, the equivalent file

(.cursorrules) was lost mid-session during the git reinitialization and was never recovered, contributing to inconsistency in the latter half of the session.

3. Annotated Claude Code Session Log

The following excerpt is from the Part 3 TDD session (Rounds 1 and 4), constructed from documented prompts and responses. The left column shows the prompt or response; the right column annotates the strategic purpose of each exchange.

Prompt / Response	Annotation
USER (Round 1, Red) "Write a failing test for <code>create_session</code>. Accept <code>user_id</code> and <code>interview_type</code> (enum: behavioral, technical, mixed), return <code>SessionCreateResponse</code>. Do not write any implementation yet. Just the test." CLAUDE Created <code>test_session_create.py</code>. Both imports fail with <code>ModuleNotFoundError</code>. <code>pytest</code>: 2 errors, 0 passed. Committed as <code>test(red)</code>. USER (Round 1, Green) "Write the minimum implementation to make that test pass. Add <code>create_session</code> to <code>session_service.py</code> and a <code>POST /api/v1/sessions</code> route. No extra logic." CLAUDE Added schema, service function, and route. 23 tests passing (22 from Part 2 + 1 new). Lint clean. Committed as <code>feat(green)</code>.	<i>The constraint "do not write any implementation yet" is load-bearing. Without it, Claude Code co-generates the implementation and produces a test that passes by construction rather than by correctness.</i> <i>Import failures constitute the red state. The commit here is the documented proof of red before green, which is what the git history must demonstrate.</i> <i>"No extra logic" prevents Claude Code from adding validation or error handling that would obscure which code caused the test to pass and complicate the refactor step.</i> <i>Verifying 23 = 22 + 1 confirms no regressions. A green state that breaks prior tests is not a valid green state.</i>
USER (Round 4, Red) "Write a failing test: if the user already has a session with status <code>'in_progress'</code>, <code>POST /sessions</code> should return 409. Do not write any implementation yet." CLAUDE Test mocks an existing <code>in_progress</code> session. <code>POST</code> returns 201 (no guard yet). Committed as <code>test(red)</code>. Implementation added <code>_check_no_active_session()</code> helper to service layer. 30 tests passing. <code>feat(green)</code>, then refactor.	<i>This is the highest-value round. The 409 guard is pure business logic with no framework support, so the failing test forces an explicit implementation decision about where the guard lives.</i> <i>Extracting the guard into a named helper was the refactor outcome. The name makes intent explicit and the helper is independently testable, the expected result of red-green-refactor applied to business logic.</i>

The central finding from this comparison is not that structured workflows produce better code, but that they produce verifiable code. The OfficeHoursQ session produced a working scaffold with no way to confirm it remains working after a change. The MockReady session ended with 30 passing tests, each a standing assertion about system behavior. That distinction, between working and verifiably working, is the practical value of the methodology demonstrated in HW4.